

REFERENCE

Tools reference

Complete reference for the tools Claude Code can use, including permission requirements and per-tool behavior.

Claude Code has access to a set of built-in tools that help it understand and modify your codebase. The tool names are the exact strings you use in [permission rules](#), [subagent tool lists](#), and [hook matchers](#). To disable a tool entirely, add its name to the `deny` array in your [permission settings](#).

To add custom tools, connect an [MCP server](#). To extend Claude with reusable prompt-based workflows, write a [skill](#), which runs through the existing `Skill` tool rather than adding a new tool entry.

Tool	Description	Permission Required
<code>Agent</code>	Spawns a subagent with its own context window to handle a task. See Agent tool behavior	No
<code>Artifact</code>	Publishes an HTML or Markdown file as an artifact : a private, interactive page on claude.ai you can share inside your organization. Requires a Team or Enterprise plan and <code>/login</code> authentication; see Availability	Yes
<code>AskUserQuestion</code>	Asks multiple-choice questions to gather requirements or clarify ambiguity	No
<code>Bash</code>	Executes shell commands in your environment. See Bash tool behavior	Yes
<code>CronCreate</code>	Schedules a recurring or one-shot prompt within the current session. Tasks are session-scoped and restored on <code>--resume</code> or <code>--continue</code> if unexpired. See scheduled tasks	No
<code>CronDelete</code>	Cancels a scheduled task by ID	No
<code>CronList</code>	Lists all scheduled tasks in the session	No
<code>Edit</code>	Makes targeted edits to specific files. See Edit tool behavior	Yes
<code>EnterPlanMode</code>	Switches to plan mode to design an approach before coding	No
<code>EnterWorktree</code>	Creates an isolated git worktree and switches into it. Pass a <code>path</code> to switch into an existing worktree of the current	No

Tool	Description	Permission Required
	repository instead of creating a new one. From within a worktree session, or from a subagent with a pinned working directory such as <code>isolation: worktree</code> , only the <code>path</code> form is available and the target must be under <code>.claude/worktrees/</code>	
<code>ExitPlanMode</code>	Presents a plan for approval and exits plan mode	Yes
<code>ExitWorktree</code>	Exits a worktree session and returns to the original directory. Not available to subagents that already run in their own working directory, such as with <code>isolation: worktree</code>	No
<code>Glob</code>	Finds files based on pattern matching. See Glob tool behavior	No
<code>Grep</code>	Searches for patterns in file contents. See Grep tool behavior	No
<code>ListMcpResourcesTool</code>	Lists resources exposed by connected MCP servers	No
<code>LSP</code>	Code intelligence via language servers: jump to definitions, find references, report type errors and warnings. See LSP tool behavior	No
<code>Monitor</code>	Runs a command in the background and feeds each output line back to Claude, so it can react to log entries, file changes, or polled status mid-conversation. See Monitor tool	Yes
<code>NotebookEdit</code>	Modifies Jupyter notebook cells. See NotebookEdit tool behavior	Yes
<code>PowerShell</code>	Executes PowerShell commands natively. See PowerShell tool for availability	Yes
<code>PushNotification</code>	Sends a desktop notification, and a phone push when Remote Control is connected, so a long-running task or scheduled task can reach you when you step away. Push delivery runs through Anthropic-hosted infrastructure, which is not accessible from Amazon Bedrock, Google Vertex AI, or Microsoft Foundry	No
<code>Read</code>	Reads the contents of files. See Read tool behavior	No
<code>ReadMcpResourceTool</code>	Reads a specific MCP resource by URI	No
<code>RemoteTrigger</code>	Creates, updates, runs, and lists Routines on claude.ai. Backs the <code>/schedule</code> command. Routines live on claude.ai and require a Pro, Max, Team, or Enterprise plan, so this tool is not accessible from Amazon Bedrock, Google Vertex AI, or Microsoft Foundry	No
<code>ScheduleWakeup</code>	Reschedules the next iteration of a self-paced /loop . Claude calls this at the end of each iteration to pick when the next one runs, between one minute and one hour out; you don't call it directly. The pending wakeup appears in <code>session_crons</code> in Stop hook input . Not available on Amazon Bedrock, Google Vertex AI, or Microsoft Foundry, where a <code>/loop</code> prompt with no interval runs on a fixed schedule instead	No

Tool	Description	Permission Required
SendMessage	Sends a message to an agent team teammate, or resumes a subagent by its agent ID. Stopped subagents auto-resume in the background. Structured team-protocol messages require agent teams	No
ShareOnboardingGuide	Uploads <code>ONBOARDING.md</code> and returns a share link teammates can open in Claude Code. Called from <code>/team-onboarding</code> after the guide is written. Available to claude.ai subscribers on Pro, Max, Team, and Enterprise plans	Yes
Skill	Executes a skill within the main conversation	Yes
TaskCreate	Creates a new task in the task list	No
TaskGet	Retrieves full details for a specific task	No
TaskList	Lists all tasks with their current status	No
TaskOutput	(Deprecated) Retrieves output from a background task. Prefer <code>Read</code> on the task's output file path	No
TaskStop	Kills a running background task by ID	No
TaskUpdate	Updates task status, dependencies, details, or deletes tasks	No
TodoWrite	Manages the session task checklist. Disabled by default as of v2.1.142 in favor of <code>TaskCreate</code> , <code>TaskGet</code> , <code>TaskList</code> , and <code>TaskUpdate</code> . Set <code>CLAUDE_CODE_ENABLE_TASKS=0</code> to re-enable	No
ToolSearch	Searches for and loads deferred tools when tool search is enabled	No
WaitForMcpServers	Waits for one or more MCP servers that are still connecting in the background, so a request can use their tools without restarting the session. Claude calls it when a needed server is not connected yet. Only appears when tool search is disabled, since <code>ToolSearch</code> handles the wait when it's enabled	No
WebFetch	Fetches content from a specified URL. See WebFetch tool behavior	Yes
WebSearch	Performs web searches. See WebSearch tool behavior	Yes
Workflow	Runs a dynamic workflow : a script that orchestrates many subagents in the background and returns one consolidated result	Yes
Write	Creates or overwrites files. See Write tool behavior	Yes

Configure tools with permission rules and hooks

For the most part, Claude decides when to use these tools and you don't need to name them yourself

when interacting with Claude. You reference tool names directly when defining permissions and other configuration:

- in `permissions.allow` and `permissions.deny` in settings, and the `/permissions` interface
- in the `--allowedTools` and `--disallowedTools` **CLI flags**
- in the Agent SDK's `allowedTools` and `disallowedTools` options
- in a **subagent's** `tools` or `disallowedTools` frontmatter
- in a **skill's** `allowed-tools` frontmatter
- in a hook's **`if` condition**

All of these accept the same rule format, `ToolName(specifier)` . The specifier depends on the tool, and several tools share a format:

Rule format	Applies to	Details
<code>Bash(npm run *)</code>	Bash, Monitor	<u>Command pattern matching</u>
<code>PowerShell(Get-ChildItem *)</code>	PowerShell	<u>Command pattern matching</u>
<code>Read(~/secrets/**)</code>	Read, Grep, Glob, LSP	<u>Path pattern matching</u>
<code>Edit(/src/**)</code>	Edit, Write, NotebookEdit	<u>Path pattern matching</u>
<code>Skill(deploy *)</code>	Skill	<u>Skill name matching</u>
<code>Agent(Explore)</code>	Agent	<u>Subagent type matching</u>
<code>WebFetch(domain:example.com)</code>	WebFetch	<u>Domain matching</u>
<code>WebSearch</code>	WebSearch	No specifier; allow or deny the tool as a whole

Tools not listed here, such as `ExitPlanMode` or `ShareOnboardingGuide` , accept only the bare tool name with no specifier.

An `Edit(...)` allow rule also grants read access to the same path, so you don't need a matching `Read(...)` rule.

Hook `matcher` fields use bare tool names, not the parenthesized rule format. See matcher patterns for the matching rules. For the field names each tool passes to `tool_input` in hooks, see the PreToolUse input reference.

Agent tool behavior

The Agent tool spawns a subagent in a separate context window. The subagent works through its task autonomously, then returns a single text result to the parent conversation. The parent does not see the subagent's intermediate tool calls or outputs, only that final result. To cap how many turns a subagent runs, set `maxTurns` in the subagent definition.

The same Agent tool also launches forked subagents when fork mode is enabled. A fork inherits the full parent conversation instead of starting fresh, always runs in the background, and still surfaces permission prompts in your terminal. The rest of this section describes named subagents.

Which tools a named subagent can use depends on the `tools` and `disallowedTools` fields in the subagent definition:

- **Neither field set:** the subagent inherits every tool available to the parent.
- `tools` **only:** the subagent gets only the listed tools.
- `disallowedTools` **only:** the subagent gets every parent tool except the listed ones.
- **Both set:** `disallowedTools` takes precedence. A tool listed in both is removed.

Launching the subagent does not itself prompt for permission. The subagent's own tool calls are checked against your permission rules as it runs:

- **Foreground subagents** show the same permission prompts you would see in the main conversation, at the moment each tool call happens.
- **Background subagents** surface permission prompts in your main session as of v2.1.186. The prompt names which subagent is asking, and pressing Esc denies that one tool call without stopping the subagent. Before v2.1.186, background subagents auto-denied any tool call that would otherwise prompt and continued without that tool.

To limit what a subagent can reach in the first place, narrow its `tools` field, leave Bash off the list, or set deny rules in your settings, as described in Control subagent capabilities. For more on choosing between foreground and background, see Run subagents in foreground or background.

Bash tool behavior

The Bash tool runs each command in a separate process with the following persistence behavior:

- When Claude runs `cd` in the main session, the new working directory carries over to later Bash commands as long as it stays inside the project directory or an additional working directory you

added with `--add-dir` , `/add-dir` , or `additionalDirectories` in settings. Subagent sessions never carry over working directory changes.

- If `cd` lands outside those directories, Claude Code resets to the project directory and appends `Shell cwd was reset to <dir>` to the tool result.
- To disable this carry-over so every Bash command starts in the project directory, set `CLAUDE_BASH_MAINTAIN_PROJECT_WORKING_DIR=1` .
- Environment variables don't persist. An `export` in one command won't be available in the next.
- Aliases and shell functions defined in your shell startup file are available. At session start, Claude Code sources `~/.zshrc` , `~/.bashrc` , or `~/.profile` depending on your shell, captures the resulting aliases, functions, and shell options, and applies them to every Bash command.

Activate your virtualenv or conda environment before launching Claude Code. To make environment variables persist across Bash commands, set `CLAUDE_ENV_FILE` to a shell script before launching Claude Code, or use a **SessionStart hook** to populate it dynamically.

Two limits bound each command:

- **Timeout:** two minutes by default. Claude can request up to 10 minutes per command with the `timeout` parameter. Override the default and ceiling with `BASH_DEFAULT_TIMEOUT_MS` and `BASH_MAX_TIMEOUT_MS` .
- **Output length:** 30,000 characters by default. When a command produces more than that, Claude Code saves the full output to a file in the session directory and gives Claude the file path plus a short preview from the start. Claude reads or searches that file when it needs the rest. Raise the limit with `BASH_MAX_OUTPUT_LENGTH` , up to a hard ceiling of 150,000 characters.

For long-running processes such as dev servers or watch builds, Claude can set

`run_in_background: true` to start the command as a background task and continue working while it runs. List and stop background tasks with `/tasks` .

Edit tool behavior

The Edit tool performs exact string replacement. It takes an `old_string` and a `new_string` and replaces the first with the second. It does not use regex or fuzzy matching.

Three checks must pass for an edit to apply:

- **Read-before-edit:** Claude must have read the file in the current conversation, and the file must

not have changed on disk since that read. This check runs first, before any string matching.

- **Match:** `old_string` must appear in the file exactly as written. A single character of whitespace or indentation difference is enough to miss.
- **Uniqueness:** `old_string` must appear exactly once. When it appears more than once, Claude either supplies a longer string with enough surrounding context to pin down one occurrence, or sets `replace_all: true` to replace them all.

Viewing a file with Bash also satisfies the read-before-edit requirement when the command is `cat` , `head` , `tail` , `sed -n 'X,Yp'` , `grep` , `egrep` , or `fgrep` on a single file with no pipes or redirects. Piped output and other Bash commands don't count, and Claude must use Read before editing in those cases.

This affects edit eligibility only, not permissions. **Read and Edit deny rules** also apply to file commands Claude Code recognizes in Bash, such as `cat` , `head` , `tail` , `sed` , and `grep` , but not to arbitrary subprocesses that read or write files indirectly, like a Python or Node script that opens files itself. The set of commands recognized for deny rules is not the same as the read-before-edit list above: for example, `egrep` and `fgrep` count for read-before-edit but are not checked against Read deny rules. For OS-level enforcement that covers every process, **enable the sandbox**.

Glob tool behavior

The Glob tool finds files by name pattern. It supports standard glob syntax including `**` for recursive directory matching:

- `**/*.js` matches all `.js` files at any depth
- `src/**/*.ts` matches all `.ts` files under `src/`
- `*.{json,yaml}` matches `.json` and `.yaml` files in the current directory

Results are sorted by modification time and capped at 100 files. If the cap is hit, Claude sees a truncation flag in the result and can narrow the pattern.

Glob does not respect `.gitignore` by default, so it finds gitignored files alongside tracked ones. This differs from **Grep**, which skips gitignored files. To make Glob respect `.gitignore` , set `CLAUDE_CODE_GLOB_NO_IGNORE=false` before launching Claude Code.

Grep tool behavior

The Grep tool searches file contents for patterns. Where **Glob** finds files by name, Grep finds lines

inside them.

Grep is built on **ripgrep** and uses ripgrep's regex syntax, not POSIX grep. Patterns that include regex metacharacters need escaping. For example, finding `interface{}` in Go code takes the pattern `interface\{\}`.

Three output modes control what comes back:

- `files_with_matches` : file paths only, no line content. This is the default.
- `content` : matching lines with file and line number.
- `count` : match count per file.

Claude can scope results by file with the `glob` parameter, such as `**/*.tsx`, or by language with the `type` parameter, such as `py` or `rust`. By default, patterns match within a single line. Claude can set `multiline: true` to match across line boundaries.

Grep respects `.gitignore`, so gitignored files are skipped. To search a gitignored file, Claude passes its path directly.

LSP tool behavior

The LSP tool gives Claude code intelligence from a running language server. After each file edit, it automatically reports type errors and warnings so Claude can fix issues without a separate build step. Claude can also call it directly to navigate code:

- Jump to a symbol's definition
- Find all references to a symbol
- Get type information at a position
- List symbols in a file
- Search for a symbol by name across the workspace
- Find implementations of an interface
- Trace call hierarchies

The tool is inactive until you install a **code intelligence plugin** for your language. The plugin bundles the language server configuration, and you install the server binary separately.

Monitor tool

ⓘ The Monitor tool requires Claude Code v2.1.98 or later.

The Monitor tool lets Claude watch something in the background and react when it changes, without pausing the conversation. Ask Claude to:

- Tail a log file and flag errors as they appear
- Poll a PR or CI job and report when its status changes
- Watch a directory for file changes
- Track output from any long-running script you point it at

Claude writes a small script for the watch, runs it in the background, and receives each output line as it arrives. You keep working in the same session and Claude interjects when an event lands. Stop a monitor by asking Claude to cancel it or by ending the session.

Monitor uses the same **permission rules as Bash**, so `allow` and `deny` patterns you have set for Bash apply here too. It is not available on Amazon Bedrock, Google Vertex AI, or Microsoft Foundry. It is also not available when `DISABLE_TELEMETRY` or `CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC` is set.

Plugins can declare monitors that start automatically when the plugin is active, instead of asking Claude to start them. See **plugin monitors**.

NotebookEdit tool behavior

NotebookEdit modifies a Jupyter notebook one cell at a time, targeting cells by their `cell_id`. It does not perform string replacement across the notebook the way **Edit** does on plain files.

Three edit modes control what happens to the target cell:

- `replace` : overwrite the cell's source. This is the default.
- `insert` : add a new cell after the target. With no `cell_id`, the new cell goes at the start of the notebook. Requires `cell_type` set to `code` or `markdown`.
- `delete` : remove the target cell.

Permission rules use the `Edit(...)` path format. A rule like `Edit(notebooks/**)` covers NotebookEdit calls on files in that directory.

PowerShell tool

The PowerShell tool lets Claude run PowerShell commands natively. On Windows, this means commands run in PowerShell instead of routing through Git Bash. On Windows without Git Bash, the tool is enabled automatically. On Windows with Git Bash installed, the tool is rolling out progressively. On Linux, macOS, and WSL, the tool is opt-in.

Enable the PowerShell tool

Set `CLAUDE_CODE_USE_POWERSHELL_TOOL=1` in your environment or in `settings.json` :

```
{
  "env": {
    "CLAUDE_CODE_USE_POWERSHELL_TOOL": "1"
  }
}
```

On Windows, set the variable to `0` to opt out of the rollout. On Linux, macOS, and WSL, the tool requires PowerShell 7 or later: install `pwsh` and ensure it is on your `PATH` .

On Windows, Claude Code auto-detects `pwsh.exe` for PowerShell 7+ with a fallback to `powershell.exe` for PowerShell 5.1. When the tool is enabled, Claude treats PowerShell as the primary shell. The Bash tool remains available for POSIX scripts when Git Bash is installed.

Claude Code spawns PowerShell with `-ExecutionPolicy Bypass` at process scope only, so `.ps1` scripts and module imports work on default Windows installs without changing the machine's policy. Process-scope bypass does not override Group Policy `MachinePolicy` or `UserPolicy` , so enterprise lockdowns still apply. To respect the machine's effective execution policy instead, set `CLAUDE_CODE_POWERSHELL_RESPECT_EXECUTION_POLICY=1` .

Shell selection in settings, hooks, and skills

Three additional settings control where PowerShell is used:

- `"defaultShell": "powershell"` in `settings.json` : routes interactive `!` commands through PowerShell. Requires the PowerShell tool to be enabled.
- `"shell": "powershell"` on individual **command hooks**: runs that hook in PowerShell. Hooks spawn PowerShell directly, so this works regardless of `CLAUDE_CODE_USE_POWERSHELL_TOOL` .

- `shell: powershell` in **skill frontmatter**: runs `!`command`` blocks in PowerShell. Requires the PowerShell tool to be enabled.

The same main-session working-directory reset behavior described under the Bash tool section applies to PowerShell commands, including the `CLAUDE_BASH_MAINTAIN_PROJECT_WORKING_DIR` environment variable.

Preview limitations

The PowerShell tool has the following known limitations during the preview:

- PowerShell profiles are not loaded
- On Windows, sandboxing is not supported

Read tool behavior

The Read tool takes a file path and returns the contents with line numbers. Claude is instructed to always pass absolute paths.

By default, Read returns the file from the start. When a whole-file read exceeds the token limit, Read returns the first page with a `PARTIAL view` notice that tells Claude how much of the file it received and how to read more with `offset` and `limit`. A read that passes an explicit `offset` or `limit` and still exceeds the token limit returns an error.

Read handles several file types beyond plain text:

- **Images:** PNG, JPG, and other image formats are returned as visual content that Claude can see, not as raw bytes. Claude Code resizes and recompresses large images to fit the model's image size limits before sending them, so Claude may see a downscaled version of a large screenshot. If Claude misses fine pixel-level detail in a large image, ask it to crop the region of interest first, for example with ImageMagick via Bash.
- **PDFs:** Claude reads short `.pdf` files whole. For PDFs longer than 10 pages, it reads in ranges with a `pages` parameter, such as `"1-5"`, up to 20 pages at a time.
- **Jupyter notebooks:** `.ipynb` files return all cells with their outputs, including code, markdown, and visualizations.

Read only reads files, not directories. Claude uses `ls` via the Bash tool to list directory contents.

WebFetch tool behavior

WebFetch takes a URL and a prompt describing what to extract. It fetches the page, converts the response to Markdown when the server returns HTML, and runs the prompt against the content using a small, fast model. For most fetches, Claude receives that model's answer, not the raw page. The conversion step is not configurable.

This makes WebFetch lossy by design. The extraction prompt determines what reaches Claude, so a result that says a page does not mention something may only mean the prompt did not ask about it. Ask Claude to fetch again with a more specific prompt, or use `curl` via Bash for the unprocessed page.

A few behaviors shape the response Claude receives:

- HTTP URLs are automatically upgraded to HTTPS.
- Large pages are truncated to a fixed character limit before processing.
- Responses are cached for 15 minutes, so repeated fetches of the same URL return quickly.
- When a URL redirects to a different host, WebFetch returns a text result that names the original URL and the redirect target instead of following it. Claude then fetches the new URL with a second WebFetch call.

In the default and `acceptEdits` permission modes, WebFetch prompts the first time it reaches a new domain, except for a built-in set of preapproved documentation domains that fetch without a prompt. To allow another domain in advance without a prompt, add a permission rule like `WebFetch(domain:example.com)`. The `auto` and `bypassPermissions` permission modes skip the prompt entirely.

An explicit `WebFetch(domain:...)` rule in `deny`, `ask`, or `allow` takes precedence over the preapproved set, so you can block a preapproved domain or require a prompt for it.

WebFetch sets a `User-Agent` header beginning with `Claude-User`, and an `Accept` header that prefers Markdown over HTML so servers that support content negotiation can return Markdown directly. Sandbox network rules are configured separately, so a domain you want a sandboxed process to reach still needs an explicit sandbox permission rule.

WebSearch tool behavior

WebSearch runs a query against Anthropic's web search backend and returns result titles and URLs. It does not fetch the result pages. To read a page Claude finds in search results, it follows up with

WebFetch.

The tool may issue up to eight backend searches per call, refining the search internally before returning results. Claude can scope results with `allowed_domains` to include only certain hosts, or `blocked_domains` to exclude them. The two lists can't be combined in a single call.

The search backend is not configurable. To search with a different provider, add an **MCP server** that exposes a search tool.

WebSearch permission rules take no specifier. A bare `WebSearch` entry in `allow` or `deny` is the only form.

❗ WebSearch is available on the Claude API and Microsoft Foundry. On Google Cloud Vertex AI it works with Claude 4 models, including Opus, Sonnet, and Haiku. Amazon Bedrock does not expose the server-side web search tool.

Write tool behavior

The Write tool creates a new file or overwrites an existing one with the full content provided. It does not append or merge.

If the target path already exists, Claude must have read that file at least once in the current conversation before overwriting it. A Write to an unread existing file fails with an error. This constraint does not apply to new files.

Viewing the file with Bash also satisfies this requirement under the same rules described in **Edit tool behavior.**

For partial changes to an existing file, Claude uses Edit instead of Write.

Check which tools are available

Your exact tool set depends on your provider, platform, and settings. To check what's loaded in a running session, ask Claude directly:

What tools do you have access to?

Claude gives a conversational summary. For exact MCP tool names, run `/mcp` .

ⓘ The **advisor tool** is a **server tool** that the API runs, rather than a tool that Claude Code implements. It has no name you can reference in permission rules or hook matchers.

See also

- **MCP servers**: add custom tools by connecting external servers
- **Permissions**: permission system, rule syntax, and tool-specific patterns
- **Subagents**: configure tool access for subagents
- **Hooks**: run custom commands before or after tool execution